Universidad Nacional de Ingeniería

Área de Conocimiento de Tecnología de la Información y Comunicación

SISTEMA DE GESTIÓN DE ALERTA (SGA)

Proyecto de Curso para la asignatura de DISEÑO DE SISTEMAS EN INTERNET

Br. Axel Iván Valle Obando
Carné: 2021-0391I
Br. Caleb Francisco Miranda Centeno
Carné: 2022-0357I
Br. Milton Joan Bell Calderón
Carné: 2021-082I
Docente: Walter Quintero

Organización: Universidad Nacional de Ingeniería (UNI), Recinto Universitario Simón Bolívar

22 de octubre de 2026

Managua, Nicaragua

ÍNDICE

RESUMEN EJECUTIVO 4

CAPÍTULO I. INTRODUCCIÓN 5

1.1. Antecedentes de la organización 5

1.2. Planteamiento del problema 5

1.3. Justificación 6

1.4. Objetivos 6

1.5. Alcances y limitaciones 7

CAPÍTULO II. MARCO TEÓRICO Y TECNOLÓGICO 8

2.1. Arquitecturas de software para la web moderna 8

2.2. Tecnologías frontend y backend 8

2.3. Evolución y estándares de servicios web (REST/GraphQL frente a SOAP legado) 9

CAPÍTULO III. ANÁLISIS Y DISEÑO DEL SISTEMA (Ingeniería Web — UWE) 10

3.1. Modelo de negocio y procesos (BPMN o similar) 10

3.2. Obtención de requisitos (casos de uso) 11

3.3. Diseño conceptual (modelo relacional y diagramas de clases) 12

3.4. Diseño navegacional (mapa del sitio y flujos de usuario) 13

3.5. Diseño de presentación (wireframes, usabilidad y accesibilidad) 14

3.6. Diseño de la infraestructura tecnológica 15

CAPÍTULO IV. IMPLEMENTACIÓN DEL SISTEMA 16

4.1. Arquitectura de tres capas aplicada al código 16

4.2. Gestión de datos y conectividad 16

4.3. Implementación de servicios web 17

4.4. Seguridad implementada 17

CAPÍTULO V. DESPLIEGUE, PRUEBAS Y SEGUIMIENTO DE CAMBIOS 19

5.1. Estrategia de despliegue (CI/CD básico y plataforma cloud) 19
5.2. Pruebas de sistema realizadas (funcionales, carga y seguridad) 20
5.3. Responsabilidad ambiental (hosting frente a servidor físico on-premise) 21
5.4. Seguimiento de cambios 21
CAPÍTULO VI. MANUAL DE USUARIO 22
6.1. Requisitos mínimos de hardware y software 22
6.2. Guía paso a paso con capturas reales del sistema 22
CAPÍTULO VII. CONCLUSIONES Y RECOMENDACIONES 29
REFERENCIAS 31
ANEXOS 32
ANEXO A. Roles y matriz de permisos 32
ANEXO B. Matriz propuesta de pruebas 32
ANEXO C. Diagramas UWE — C.1. Casos de uso de procesos 34
C.2. Casos de uso navegacionales 35
C.3. Modelo de contenido 36
C.4. Modelo de navegación 37
ANEXO D. Prototipos visuales recibidos 38
D.1. Inicio de sesión 38
D.2. Dashboard 39
D.3. Gestión de personas 40
D.4. Horarios 41
D.5. Registro 42
D.6. Reportes 43
ANEXO E. Enlaces, API y despliegue Docker 44

RESUMEN EJECUTIVO

El proyecto final se denomina Sistema de Gestión de Alerta. La versión ejecutable documentada se identifica actualmente en la interfaz como Sistema de Gestión de Asistencia (SGA) y permite administrar personas, horarios, registros de asistencia, incidencias y justificaciones en una aplicación web. La organización de referencia académica es la Universidad Nacional de Ingeniería (UNI); aún se debe acordar con la UNI la unidad operativa piloto y su persona de contacto.

El repositorio de trabajo contiene una interfaz Next.js con panel de resumen y módulos de personas, horarios, asistencia, incidencias y reportes, además de una API REST en Django REST Framework y modelos relacionales preparados para PostgreSQL. Para completar la fase 3 se incorporó inicio de sesión con JWT en cookies HttpOnly, protección CSRF, renovación y revocación de sesión, grupos de rol y permisos de solo lectura para supervisión y auditoría. Se preparó Docker Compose con frontend, API, base de datos PostgreSQL y Caddy como proxy HTTPS.

El entorno Docker local se ejecutó correctamente. Para el manual se capturaron después siete pantallas reales de la aplicación pública en Render, con un filtro que evita mostrar datos personales en Personas. El 22 de octubre de 2026 quedaron accesibles la interfaz y la API por HTTPS; el administrador inicial ingresó al panel. Se implementaron reportes, configuración de cuentas y roles, auditoría e historial personal. Aún no existe un piloto institucional ni un informe de pruebas funcionales, de carga o de seguridad.

Antes de la defensa, el equipo debe comprender y explicar cada cambio, documentar las pruebas pendientes y acordar con la UNI la unidad piloto. La dirección pública es https://sga-cal-miranda-web.onrender.com/login. El plan gratuito puede suspender los servicios por inactividad, por lo que conviene abrirlos antes de la exposición (Universidad Nacional de Ingeniería, 2026a).

CAPÍTULO I. INTRODUCCIÓN

1.1. Antecedentes de la organización

La Universidad Nacional de Ingeniería es la institución académica en cuyo contexto se presenta este proyecto de la asignatura Diseño de Sistemas en Internet, impartida en el Recinto Universitario Simón Bolívar. De acuerdo con la decisión del equipo, la UNI se nombra como organización de referencia. La documentación disponible no identifica una facultad, área administrativa o unidad específica que haya solicitado el sistema; por ello, la UNI no se presenta como cliente institucional que haya validado requisitos ni como piloto confirmado.

El diagnóstico de las fases iniciales describe una necesidad frecuente en instituciones educativas y organizaciones con personal: el control de asistencia puede apoyarse en listas y hojas de cálculo separadas, lo que complica la búsqueda histórica y la consolidación (Equipo SGA, 2026a). Esta descripción es el escenario de partida del proyecto y aún debe contrastarse mediante entrevista, observación y revisión de registros de una unidad piloto.

1.2. Planteamiento del problema

La gestión manual o dispersa de la asistencia puede producir inconsistencias, duplicidad, demora en las consultas y trabajo adicional al preparar resúmenes. Si no se centralizan los datos y los permisos, también resulta difícil mantener un historial coherente y limitar el acceso a la información según la función del usuario. No se dispone todavía de mediciones de tiempo, tasas de error ni registros de una unidad de la UNI; las causas y efectos se consideran hipótesis de diagnóstico que deberán comprobarse con el piloto.

Elemento	Descripción del diagnóstico inicial
Causas a validar	Listas físicas o archivos independientes; transcripción manual; ausencia de validaciones compartidas; permisos poco diferenciados.
Efectos posibles	Datos inconsistentes o duplicados; consultas históricas lentas; consolidación manual; dificultad para revisar tardanzas, ausencias e incidencias.

Elemento	Descripción del diagnóstico inicial
Evidencia disponible	Planteamiento documentado en la Fase 1. Aún no hay acta de entrevista, observación ni mediciones del proceso de una unidad piloto.

1.3. Justificación

Una aplicación web con interfaz separada de una API permite centralizar registros y aplicar validaciones en el servidor. El SGA brinda un punto único para mantener personas, horarios, asistencia, incidencias y justificaciones; el panel resume información consultada desde la API. El diseño de API REST facilita el intercambio entre frontend y backend y permite documentar operaciones con OpenAPI/Swagger.

La ventaja competitiva propuesta consiste en un flujo web sencillo, datos relacionados y permisos en el servidor, en lugar de depender de archivos aislados. Se trata de una ventaja esperada y no de un resultado medido: aún no se han comparado tiempos reales ni se ha ejecutado un piloto. El despliegue contenerizado puede reducir pasos de instalación y permitir escoger después un proveedor sencillo, comparando costo mensual, respaldos, TLS, actualizaciones y esfuerzo operativo.

1.4. Objetivos

Objetivo general

Desarrollar y documentar un sistema web de gestión de asistencia que centralice personas, horarios, registros, incidencias y justificaciones, y que proteja los recursos mediante autenticación y autorización por rol.

Objetivos específicos

OE1. Analizar el proceso de asistencia de una unidad piloto de la UNI y confirmar sus actores, necesidades y restricciones.

OE2. Modelar requisitos, procesos, contenido y navegación con los artefactos UWE preparados para el proyecto.

OE3. Implementar una interfaz web conectada a una API REST y una base de datos relacional para los procesos incluidos en el alcance.

OE4. Proteger el acceso con sesiones JWT en cookies seguras, controles CSRF y autorización basada en grupos de rol.

OE5. Preparar el despliegue contenerizado y documentar las pruebas, la publicación y la operación que deben completarse antes de la defensa.

1.5. Alcances y limitaciones

El alcance del proyecto distingue entre capacidades que aparecen en el código, componentes diseñados y trabajo que requiere ejecución o validación. La inspección del repositorio se realizó el 21 de octubre de 2026; describe archivos presentes, no resultados de ejecución en producción.

Estado	Alcance
Implementado en el código observado	Login y cierre de sesión; JWT con renovación y revocación; permisos por grupos; CRUD de personas, horarios, asistencia, incidencias y justificaciones; panel de resumen; API REST y esquema OpenAPI/Swagger.
Autohospedaje preparado	Docker Compose con frontend Next.js, API Django, PostgreSQL y Caddy para una instalación propia. Dominio, servidor, DNS, secretos y respaldos deben configurarse en ese destino. La demostración pública ya está desplegada en Render y Neon.
Implementado parcialmente	El módulo /reportes consulta asistencia por período, grupo o área, rol y estado, e imprime o exporta CSV. Se incorporaron configuración de cuentas y roles, bitácora de eventos e historial propio vinculado a Persona. Siguen fuera del alcance la carga masiva, SSO/OAuth2, las integraciones externas, el aislamiento por sede y la validación con la organización piloto.
Pendiente de evidencia	Pruebas funcionales, de permisos, carga, seguridad y restauración documentadas; piloto y aceptación de usuarios; pipeline CI/CD; medición ambiental comparable.

Este manual muestra capturas reales de la versión de producción publicada en Render y conectada a Neon. Docker Compose se conserva como alternativa para ejecutar el sistema localmente o migrarlo a infraestructura propia.

CAPÍTULO II. MARCO TEÓRICO Y TECNOLÓGICO

2.1. Arquitecturas de software para la web moderna

Una arquitectura monolítica puede empaquetar interfaz, lógica y datos en una sola aplicación desplegable. Una arquitectura de microservicios divide capacidades en servicios independientes con operación y contratos propios, a costa de mayor complejidad de red, observabilidad, despliegue y coordinación. El SGA adopta una aplicación web modular de tres capas: frontend Next.js, API Django REST Framework y base PostgreSQL. No es una arquitectura de microservicios; esta opción mantiene límites claros entre capas sin añadir infraestructura que el proyecto todavía no necesita.

Alternativa	Ventaja	Costo / límite	Decisión para SGA
Monolito tradicional	Menor cantidad de componentes operativos.	Puede acoplar presentación y backend si no se separan las responsabilidades.	No se usa como aplicación única; se exige separación frontend/API.
Monolito modular de tres capas	Separación de responsabilidades y despliegue sencillo con Docker Compose.	Frontend y backend se construyen y configuran por separado.	Elegido para el tamaño y alcance académico actual.
Microservicios	Escalado y liberación independiente por capacidad.	Necesita más monitoreo, redes, seguridad y administración.	No justificado por el volumen ni por un equipo operativo disponible.

2.2. Tecnologías frontend y backend

Componente	Tecnología del proyecto	Uso y razón
Frontend	Next.js 16.4.2, React 19.2.8 y TypeScript	Interfaz web por rutas; formularios y vistas cliente; tipos para datos de API.
Backend/API	Python 3.12, Django 5.2.17 y Django REST Framework 3.18.1	Autenticación, reglas de negocio, validación y servicios REST.
Persistencia	PostgreSQL 17 en contenedor	Base relacional con claves foráneas, restricciones y migraciones.
Documentación de API	drf-spectacular y Swagger UI	Esquema OpenAPI generado desde rutas y serializadores.
Contenedores y proxy	Docker Compose, Caddy 2	Orquestación de los servicios y terminación HTTPS al configurarse un dominio público.

La selección separa la experiencia del usuario del servicio que valida y persiste los datos. El ORM de Django evita construir manualmente la mayoría de las consultas; las migraciones versionan el esquema. Los requisitos reales de operación, disponibilidad y capacidad deberán medirse con la organización piloto antes de dimensionar un servidor.

2.3. Evolución y estándares de servicios web (REST/GraphQL frente a SOAP legado)

El SGA expone una API REST que representa personas, horarios, asistencias, incidencias y justificaciones mediante recursos HTTP. Las solicitudes y respuestas de datos usan JSON; GET consulta, POST crea, PUT reemplaza, PATCH modifica parcialmente y DELETE elimina. El esquema OpenAPI y la interfaz Swagger sirven para explorar el contrato (Equipo SGA, 2026b).

SOAP usa contratos XML y convenciones asociadas como WSDL; GraphQL permite pedir conjuntos de datos seleccionados a través de un esquema y un punto de entrada. Para el alcance actual, REST resulta directo de implementar y documentar con la pila Django ya disponible. No se ha integrado GraphQL ni un servicio externo. La existencia de documentación Swagger no sustituye la prueba de cada endpoint.

CAPÍTULO III. ANÁLISIS Y DISEÑO DEL SISTEMA (Ingeniería Web — UWE)

3.1. Modelo de negocio y procesos (BPMN o similar)

El proceso objetivo parte de la administración de personas y horarios. Un gestor registra un evento de asistencia para una persona y una jornada; si existe tardanza o ausencia puede registrarse una incidencia. La justificación asociada se revisa y, si se aprueba, el registro pasa a estado justificado. Los usuarios con permisos de lectura consultan los datos; el panel resume cantidades almacenadas. El flujo es una síntesis del alcance diseñado, no una descripción validada de un procedimiento vigente en una unidad de la UNI.

Actor	Responsabilidad del proceso objetivo	Estado en la aplicación
Administrador	Crea cuentas y asigna grupos; mantiene la configuración.	Cuenta de Django y grupos; administración de usuarios y roles en /configuracion.
Gestor de asistencia	Mantiene catálogos y registra asistencia e incidencias.	Puede leer y escribir los recursos de negocio por API.
Supervisor / Docente	Consulta información para seguimiento.	Grupo de solo lectura para API; ámbito organizacional no filtrado.
Auditor / Dirección	Consulta datos para revisión.	Grupo de solo lectura; consulta de la bitácora sin permisos para modificarla.
Empleado / Estudiante	Consulta su propio historial o solicita justificación.	Consulta de historial propio; requiere una cuenta vinculada a una ficha Persona activa.

El proceso propuesto considera la justificación como una entidad vinculada a una incidencia. La revisión aprobada actualiza la asistencia a estado justificada. Las reglas de aprobación, quién puede solicitarla y quién puede cambiarla deben confirmarse con la unidad piloto antes de producción.

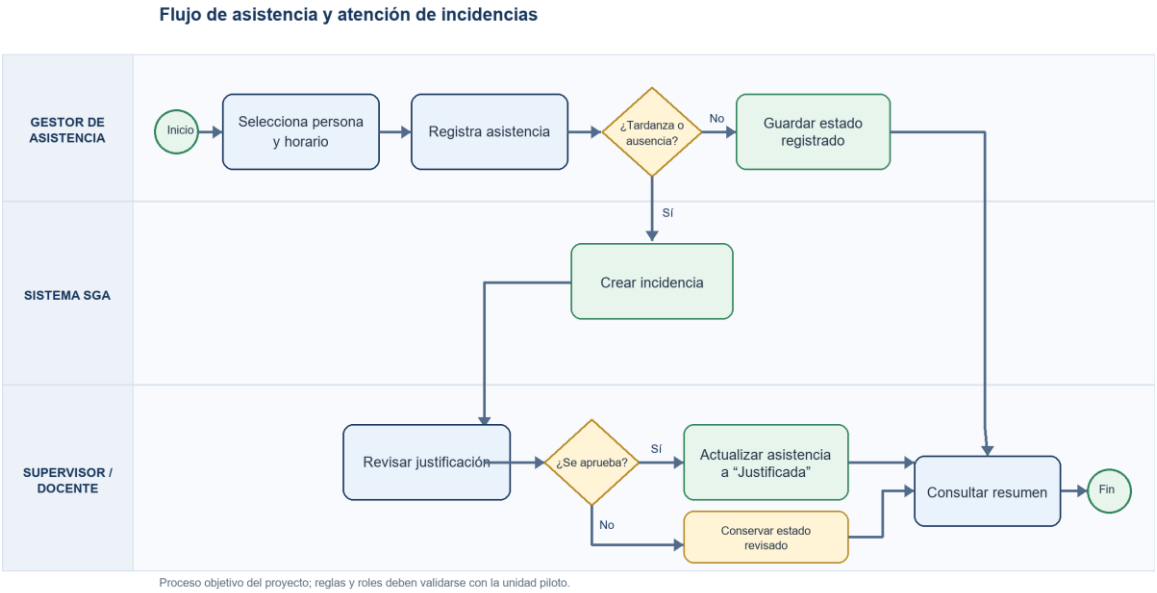


Figura 3.1. Modelo del proceso propuesto para registrar asistencia y atender incidencias. Nota. Flujo de diseño pendiente de validación con la unidad piloto.

3.2. Obtención de requisitos (casos de uso)

La Fase 1 definió actores y necesidades preliminares y la carpeta del proyecto contiene diagramas de casos de uso UWE para procesos y navegación (Equipo SGA, 2026a). La evidencia no registra entrevistas reales; por tanto, los requisitos siguientes se presentan como alcance de diseño y estado observado del código. La retroalimentación docente de la fase inicial señaló como aspectos por reforzar la formalización UWE, los wireframes y el análisis ambiental; esta entrega incorpora esos apartados como material de diseño, mientras la validación operativa sigue pendiente (Equipo SGA, 2026c).

ID	Requisito funcional	Estado
RF-01	Autenticar usuario autorizado y cerrar sesión.	Implementado en código; pruebas funcionales pendientes.
RF-02	Administrar personas con código único, nombres, apellidos y estado.	Implementado en interfaz y API.
RF-03	Crear horarios con nombre, entrada, salida y tolerancia.	Implementado; valida salida posterior a entrada.
RF-04	Registrar asistencia vinculada a persona, horario, fecha y estado.	Implementado; unicidad por persona, horario y fecha.
RF-05	Registrar incidencias y crear una justificación vinculada.	Implementado; incluye estados y revisión.

ID	Requisito funcional	Estado
RF-06	Mostrar resumen de personas, horarios y asistencias.	Implementado en Dashboard; no es un reporte parametrizable.
RF-07	Generar reportes por período y mantener bitácora de auditoría.	Reportes y bitácora implementados; las pruebas funcionales y de permisos están pendientes.
RF-08	Permitir al empleado consultar su historial propio.	Implementado en /mi-historial; requiere vínculo activo usuario-Persona. Pruebas pendientes.

ID	Requisito no funcional	Tratamiento actual
RNF-01	Acceso autenticado a los datos de negocio.	Cookie JWT y permisos de servidor.
RNF-02	Proteger operaciones de escritura frente a CSRF.	Token CSRF y cabecera X-CSRFToken en solicitudes inseguras.
RNF-03	Cifrar la conexión pública.	HTTPS activo en Render. Caddy queda preparado para TLS automático en una instalación propia con dominio y DNS.
RNF-04	Limitar intentos y administrar contraseñas.	Límite de login 5/minuto; validadores Django con mínimo de 12 caracteres.
RNF-05	Restringir operaciones según rol.	Grupos de escritura y lectura; pruebas de permisos pendientes.
RNF-06	Facilitar operación y recuperación.	Compose y volumen PostgreSQL; política de respaldo aún debe decidirse.

3.3. Diseño conceptual (modelo relacional y diagramas de clases)

El modelo persistente incluye Persona, Horario, RegistroAsistencia, Incidencia, Justificacion y AuditEvent. La ficha Persona mantiene una relación opcional uno a uno con la cuenta de Django para habilitar la consulta personal. La bitácora conserva el actor, la entidad, la acción, la fecha y los nombres de los campos modificados, sin guardar contraseñas ni valores anteriores.

Entidad	Atributos / relaciones relevantes	Restricción principal
Persona	código, nombres, apellidos, activo; relacionada con registros de asistencia.	Código único.
Horario	nombre, hora_entrada, hora_salida, tolerancia_minutos.	Nombre único; salida posterior a entrada validada por serializador.

Entidad	Atributos / relaciones relevantes	Restricción principal
RegistroAsistencia	persona, horario, fecha, hora, estado, observación.	Único por persona + horario + fecha; persona inactiva no se admite.
Incidencia	asistencia, tipo, descripción, estado.	Pertenece a un registro de asistencia.
Justificacion	incidencia, motivo, estado, observación y fechas de revisión.	Relación uno a uno con incidencia.
Usuario / Grupo	cuenta de Django y pertenencia a grupos de rol.	Aún no existe vínculo directo con Persona ni alcance por área.

Las relaciones protegen la integridad referencial: la asistencia referencia persona y horario, la incidencia referencia asistencia y la justificación corresponde a una incidencia. Persona se vincula opcionalmente uno a uno con una cuenta de Django. AuditEvent registra actor, entidad, acción, identificador, fecha y nombres de campos modificados; no guarda contraseñas ni valores anteriores. No se modelan sedes ni grupos organizativos, por lo que ese aislamiento requiere definir el alcance con la unidad piloto.

3.4. Diseño navegacional (mapa del sitio y flujos de usuario)

El flujo local parte de /login y valida sesión y rol en el servidor. Administrador y Gestor usan los módulos operativos; Supervisor / Docente consulta; Administrador gestiona cuentas y configuración; Administrador y Auditor / Dirección consultan la bitácora. Empleado / Estudiante accede únicamente a /mi-historial cuando su cuenta está vinculada a una ficha Persona activa.

Ruta	Vista	Estado
/login	Inicio de sesión	Implementada.
/dashboard	Panel de resumen	Implementado; datos calculados sobre registros disponibles.
/	Personas	Implementada.
/horarios	Horarios	Implementada.
/asistencias	Registros de asistencia	Implementada.
/incidencias	Incidencias y justificaciones	Implementada.
/reportes, /auditoria, /mi-historial	Vistas objetivo en el diseño	/reportes, /auditoria y /mi-historial están implementadas y protegidas por permisos de rol.

3.5. Diseño de presentación (wireframes, usabilidad y accesibilidad)

Las imágenes de login, dashboard, gestión, horarios, registro y reportes recibidas se incorporan en el Anexo D como prototipos visuales. No son capturas de una aplicación desplegada: algunas usan la marca ficticia “Universidad del Futuro”, contienen datos ilustrativos o presentan funciones que no existen en el código. El login implementado del SGA utiliza usuario y contraseña; no ofrece recuperación automática ni acceso Microsoft/Google.

La interfaz implementada usa campos y controles HTML convencionales. El formulario de login tiene etiquetas asociadas, mensajes de error y estados de carga. Para cerrar criterios de accesibilidad, el equipo debe revisar navegación por teclado, contraste, foco visible, lector de pantalla y diseño en móviles; aún no hay informe formal de accesibilidad ni prueba con usuarios.

3.6. Diseño de la infraestructura tecnológica

La arquitectura local conserva Docker Compose con Caddy, Next.js, Django y PostgreSQL para desarrollo y demostración. La publicación real usa dos servicios Docker gratuitos en Render: la interfaz Next.js redirige /api/, /admin/ y /static/ hacia Django, mientras PostgreSQL se aloja en Neon, en la región AWS US East 1. Render entrega HTTPS a ambos servicios. Las credenciales y la cadena de conexión se configuran como variables privadas, fuera del repositorio.

Componente	Responsabilidad	Ruta / conexión
Navegador	Interfaz de usuario.	HTTPS al dominio que se configure.
Caddy	Proxy inverso y TLS.	/api/*, /admin/* y /static/* a Django; demás rutas a Next.js.
Next.js	Frontend web en contenedor.	Puerto interno 3000; no se publica directamente al exterior.
Django REST	Autenticación, permisos, API y ORM.	Puerto interno 8000; corre como usuario sin privilegios y sólo es accesible desde red Docker/proxy.
PostgreSQL	Persistencia relacional.	Puerto interno 5432; volumen sga_database.

No se ha elegido proveedor, tamaño de máquina ni plan de respaldos. El archivo .env.example contiene marcadores y no debe usarse con secretos de ejemplo en producción. El Anexo E describe el inicio del stack para que el equipo valore después opciones de bajo mantenimiento.

CAPÍTULO IV. IMPLEMENTACIÓN DEL SISTEMA

Esta sección describe la implementación incorporada al repositorio hasta el 22 de octubre de 2026. Además del arranque local con Docker, la interfaz y la API quedaron publicadas en Render y conectadas a Neon. Los chequeos de disponibilidad y el acceso inicial del administrador confirman el despliegue, pero no equivalen a una evaluación funcional, de carga o de seguridad exhaustiva.

4.1. Arquitectura de tres capas aplicada al código

Capa	Archivos / tecnología	Responsabilidad
Presentación	frontend/src/app y componentes; Next.js, React, TypeScript.	Rutas, formularios, tablas, mensajes, navegación y consumo de API.
Aplicación / API	backend/config y apps Django; Django REST Framework.	Autenticación, autorización, serialización, validación y reglas de negocio.
Datos	PostgreSQL y Django ORM.	Persistencia de cuentas, personas, horarios, asistencia, incidencias y justificaciones.

La API es el contrato de integración entre las capas. El cliente web llama las rutas con credenciales de cookie y CSRF. El backend valida usuario, rol y datos antes de consultar o actualizar la base. El proxy expone un solo dominio en la configuración propuesta.

4.2. Gestión de datos y conectividad

Django ORM mapea modelos a tablas y utiliza migraciones para versionar cambios de esquema. El contenedor backend ejecuta migraciones al arrancar y sirve la aplicación con Gunicorn. PostgreSQL usa variables de entorno DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT y DB_SSLMODE; la clave de Django, los hosts permitidos y orígenes CORS/CSRF también se configuran por entorno.

El despliegue creó el administrador inicial mediante variables privadas de Render y aplicó las migraciones en Neon. La cuenta debe administrarse desde el sistema; la cadena de conexión y las contraseñas no forman parte del repositorio ni de este documento. Antes de operar con datos institucionales, se necesita una política de respaldos y restauración verificada.

4.3. Implementación de servicios web

Recurso	Rutas	Métodos principales	Formato
Autenticación	/api/auth/csrf/, /login/, /refresh/, /logout/, /me/	GET y POST según acción.	JSON y cookies HttpOnly.
Personas	/api/personas/ y /api/personas/{id}/	GET, POST, PUT, PATCH, DELETE.	JSON.
Horarios	/api/horarios/ y /api/horarios/{id}/	GET, POST, PUT, PATCH, DELETE.	JSON.
Asistencias	/api/asistencias/ y /api/asistencias/{id}/	GET, POST, PUT, PATCH, DELETE.	JSON.
Incidencias	/api/incidencias/ y /api/incidencias/{id}/	GET, POST, PUT, PATCH, DELETE.	JSON.
Justificaciones	/api/justificaciones/ y /api/justificaciones/{id}/	GET, POST, PUT, PATCH, DELETE.	JSON.
Documentación	/api/schema/ y /api/docs/	GET.	OpenAPI y Swagger UI.

Las rutas de negocio siguen el enrutador de Django REST Framework. Swagger está disponible en https://sga-cal-miranda-web.onrender.com/api/docs/ y el esquema OpenAPI en https://sga-cal-miranda-web.onrender.com/api/schema/; ambas rutas también se encuentran en el dominio de la API. En local, Swagger se consulta en http://localhost:8000/api/docs/ con el backend activo. La documentación no sustituye las pruebas de cada endpoint.

4.4. Seguridad implementada

Control	Implementación en el código	Alcance / consideración
Autenticación	JWT: acceso 10 minutos, renovación 1 día, rotación de refresh y lista de revocación; cookies sga_access y sga_refresh HttpOnly.	El JWT no equivale a OAuth2 ni a SSO. No hay MFA.
CSRF	Token previo en /api/auth/csrf/; protección en login, renovación, cierre y métodos de escritura; cliente envía X-CSRFToken.	La protección debe comprobarse en pruebas de integración.
Contraseñas	Validadores Django: similitud de atributos, mínimo de 12 caracteres, claves comunes y sólo numéricas.	No hay flujo público de registro ni restablecimiento de contraseña.

Control	Implementación en el código	Alcance / consideración
Permisos	Administrador y Gestor escriben; Supervisor/Docente y Auditor/Dirección sólo leen; usuario sin grupo permitido no inicia sesión.	No hay alcance por sede, área o persona; requiere definición organizacional.
Limitación de intentos	Login: 5 solicitudes por minuto; API general: 120/min anónimo y 300/min autenticado.	El límite de DRF requiere observabilidad y política de caché adecuadas al escalar.
CORS y CSRF	Orígenes permitidos definidos por variables de entorno; solicitudes con credenciales; rutas CORS limitadas a /api/.	En producción se debe configurar sólo el origen público real.
HTTPS y cookies	Secure fuera de DEBUG, redirección SSL, HSTS por defecto en producción y proxy Caddy con TLS automático.	Depende de configurar dominio, DNS, firewall y certificado.
XSS y encabezados	React escapa texto renderizado por defecto; CSP y encabezados de seguridad en Next.js/Django.	La CSP conserva unsafe-inline por compatibilidad de Next.js; no elimina XSS y debe probarse.
Secretos y hosts	SECRET_KEY obligatoria en producción; DEBUG desactivado y ALLOWED_HOSTS configurable.	Rotar secretos de ejemplo y verificar la lista de comprobación de despliegue Django.
Contenedores	Backend con usuario Linux sin privilegios; .dockerignore excluye archivos .env y dependencias locales; Compose supervisa salud básica de API y frontend.	Actualizar imágenes, revisar alertas y configurar copias de seguridad sigue siendo responsabilidad operativa.

La autenticación y autorización de API se incorporaron siguiendo controles disponibles en Django, DRF y SimpleJWT (Django REST Framework, s. f.-a, s. f.-b; Django REST Framework Simple JWT, s. f.-a, s. f.-b; Django Software Foundation, s. f.-a, s. f.-b). No se reporta una auditoría de seguridad, pentest ni prueba de intrusión. Se recomienda revisar las configuraciones de producción con la lista oficial de Django antes de publicar (Django Software Foundation, s. f.-a).

CAPÍTULO V. DESPLIEGUE, PRUEBAS Y SEGUIMIENTO DE CAMBIOS

5.1. Estrategia de despliegue (CI/CD básico y plataforma cloud)

El 22 de octubre de 2026 se desplegaron dos servicios Docker gratuitos en Render desde la rama main de https://github.com/CalMiranda/SGA y una base PostgreSQL Free en Neon. La interfaz pública es https://sga-cal-miranda-web.onrender.com/login y la API está en https://sga-cal-miranda-api.onrender.com. Render construyó las imágenes; después de los cambios integrados por pull request se iniciaron manualmente las publicaciones correspondientes. No se configuró un pipeline CI/CD ni un dominio propio. El entorno Docker Compose continúa disponible para uso local. La tabla siguiente se refiere exclusivamente a una futura migración a un servidor propio; no describe el despliegue público actual.

Etapa (servidor propio)	Acción de migración	Estado
Preparar	Elegir VPS o servicio compatible con Docker; definir presupuesto, región, respaldos y responsable operativo.	Pendiente solo si el equipo migra a hosting propio.
Configurar	Crear .env productivo con dominio, orígenes permitidos, SECRET_KEY y contraseña aleatoria de PostgreSQL.	Para Render + Neon ya está configurado; se repite si hay migración.
Publicar	Apuntar DNS, abrir 80/443 y ejecutar docker compose up --build -d.	No aplica al despliegue actual; pendiente solo para un servidor propio.
Inicializar	Crear superusuario, asignar grupo y comprobar migraciones y HTTPS.	Render + Neon están activos; falta asignar responsable, respaldos y plan operativo.
Mantener	Probar restauración de respaldos, actualizar imágenes y revisar logs y alertas.	Plan operativo debe asignarse.

El servicio público usa HTTPS provisto por Render. Los dos servicios gratuitos pueden suspenderse después de un período de inactividad, lo que retrasa la primera solicitud; además, su almacenamiento local es temporal. Los datos relacionales persisten en Neon. Este despliegue permite la defensa académica, pero requiere vigilancia de límites gratuitos, respaldos y un plan de operación antes de un uso institucional continuo.

5.2. Pruebas de sistema realizadas (funcionales, carga y seguridad)

El 22 de octubre de 2026 se verificó que la página /login, el endpoint /api/auth/csrf/ y el esquema /api/schema/ respondieran con HTTP 200 desde el dominio público. Render informó despliegue exitoso de frontend y backend; Django aplicó las migraciones en Neon y creó el administrador inicial. El usuario confirmó que ingresó al panel. Estas comprobaciones de disponibilidad y acceso no sustituyen una matriz de pruebas funcionales, de permisos, CSRF, carga, seguridad o restauración; esas evidencias aún deben registrarse con fecha, ambiente, datos y responsable.

Componente	Estado observado	Chequeo configurado
PostgreSQL 17	Healthy	pg_isready
API Django REST	Healthy	GET /api/schema/
Frontend Next.js	Healthy	GET /login
Tipo	**Verificación propuesta**	**Estado**
Funcional	Login válido, clave inválida, cuenta sin grupo y cierre de sesión.	Pendiente de ejecutar.
Autorización	Anonimato rechazado; perfiles de lectura reciben 403 al intentar escribir; perfiles de gestión escriben.	Pendiente de ejecutar.
Reglas de negocio	Código de persona duplicado; horario inválido; asistencia duplicada; justificación ya existente y revisión aprobada/rechazada.	Pendiente de ejecutar.
CSRF / navegador	Escritura sin token y con token; vencimiento/renovación JWT; cookies HttpOnly, Secure y SameSite en HTTPS.	Pendiente de ejecutar.
CORS / XSS	Origen permitido y denegado; contenido con caracteres HTML; CSP y encabezados en respuestas web.	Pendiente de ejecutar.
Carga	Medir lectura y escritura bajo un volumen representativo acordado con el piloto.	Pendiente; no hay carga estimada.
Recuperación	Restaurar una copia de PostgreSQL y confirmar integridad de registros.	Pendiente de definir y ejecutar.

Los archivos visuales de prototipo no se consideran evidencia de pruebas ni de funcionamiento. El grupo debe conservar capturas reales, solicitudes/respuestas de API y resultados de cada caso después de ejecutarlo.

5.3. Responsabilidad ambiental (hosting frente a servidor físico on-premise)

La alternativa elegida utiliza dos servicios compartidos de Render que se suspenden cuando no reciben tráfico y una base de datos administrada en Neon. Frente a un servidor físico propio, evita mantener una máquina encendida para esta demostración y reduce tareas locales de operación. No se dispone de mediciones de energía, emisiones o consumo equivalente, por lo que no se afirma un ahorro cuantificado. El equipo debe vigilar el uso de recursos, la retención de datos y el costo si supera los límites gratuitos.

Después del piloto se recomienda registrar páginas impresas por período, tiempo administrativo, uso medio de CPU/RAM y política de retención. Esos datos permitirán comparar la alternativa de hosting con el proceso previo, sin asumir valores de eficiencia que aún no se conocen.

Criterio	Render + Neon (planes gratuitos)	Servidor físico on-premise
Uso de energía	No se dispone de una medición de energía o emisiones atribuible a este proyecto; los recursos del proveedor son compartidos.	Consumo y emisiones dependen del equipo, carga y refrigeración; no hay mediciones de un servidor de referencia de la UNI.
Operación	No se administra hardware local. Render puede suspender servicios por inactividad; Neon mantiene la base. El equipo debe planificar respaldos y restauración.	La institución asume energía, climatización, espacio, mantenimiento, reemplazo y respaldos.
Evidencia disponible	Alternativa elegida para la demostración; no hay medición comparable de kWh ni CO_2e.	No se dispone de mediciones del proceso o equipo local.

5.4. Seguimiento de cambios

Los cambios de seguridad, módulos, Docker y documentación se integraron en https://github.com/CalMiranda/SGA. El despliegue Render y Neon se incorporó mediante el pull request 3; las correcciones del proxy público, mediante los pull requests 4 y 5; y el informe final con capturas de producción, mediante el pull request 6. Los commits de esas actualizaciones figuran con autor Axel Valle. Este informe no atribuye automáticamente a los demás integrantes cambios que no consten en el historial; cada miembro debe revisar y poder defender sus aportes conforme a la guía (Universidad Nacional de Ingeniería, 2026a).

CAPÍTULO VI. MANUAL DE USUARIO

Este manual describe el entorno Docker local e incluye capturas reales del sistema en ese entorno. La versión en producción, la URL definitiva y las capturas de producción quedan pendientes hasta seleccionar y configurar el hosting.

6.1. Requisitos mínimos de hardware y software

Perfil	Requisitos
Usuario final	Computadora o teléfono con navegador actualizado (Chrome, Edge, Firefox o Safari), conexión a la red donde esté publicado el SGA y credenciales habilitadas.
Equipo de desarrollo local	Windows, macOS o Linux; Python 3.12; Node.js 22; PostgreSQL 17, o Docker Engine con Docker Compose; 8 GB de RAM recomendados para ejecutar servicios y herramientas.
Servidor futuro	Host compatible con Docker Compose, dominio y DNS, acceso a puertos 80/443, almacenamiento persistente, respaldo PostgreSQL y secretos de producción fuera del código.

6.2. Guía paso a paso con capturas reales del sistema

Las figuras 6.1 a 6.7 son capturas reales de la aplicación publicada en Render el 22 de octubre de 2026. La vista de Personas se filtró para no mostrar nombres ni códigos institucionales; las demás pantallas reflejan el estado inicial de la base de datos en Neon, sin horarios, asistencias ni incidencias registrados. La dirección pública es https://sga-cal-miranda-web.onrender.com/login. El módulo de API también cuenta con Swagger público en /api/docs/ y el esquema OpenAPI en /api/schema/.

A. Acceso al sistema publicado

Paso 1. Abra un navegador actualizado y visite https://sga-cal-miranda-web.onrender.com/login. El equipo necesita conexión a internet; no requiere instalar Docker para utilizar el servicio público.

Paso 2. Si la página tarda en responder tras un período sin uso, espere a que los servicios gratuitos de Render se reactiven y vuelva a cargarla. Confirme que el navegador muestra una conexión HTTPS.

Paso 3. Solicite al administrador una cuenta autorizada para el rol que utilizará. No use la cuenta administradora inicial para las actividades ordinarias del piloto.

Paso 4. Mantenga la contraseña privada y cierre la sesión al terminar, especialmente en equipos compartidos.

B. Habilitación de una cuenta

Paso 5. Inicie sesión como Administrador y abra https://sga-cal-miranda-web.onrender.com/configuracion. Desde esa pantalla puede crear o editar cuentas, asignar un rol, activar o desactivar el acceso y vincular cada cuenta con su ficha de Persona.

C. Uso del sistema

Paso 6. En /login escriba el usuario y la contraseña asignados. El sistema valida la sesión y abre el Dashboard. Si las credenciales son incorrectas o la cuenta carece de rol, solicite revisión al administrador.

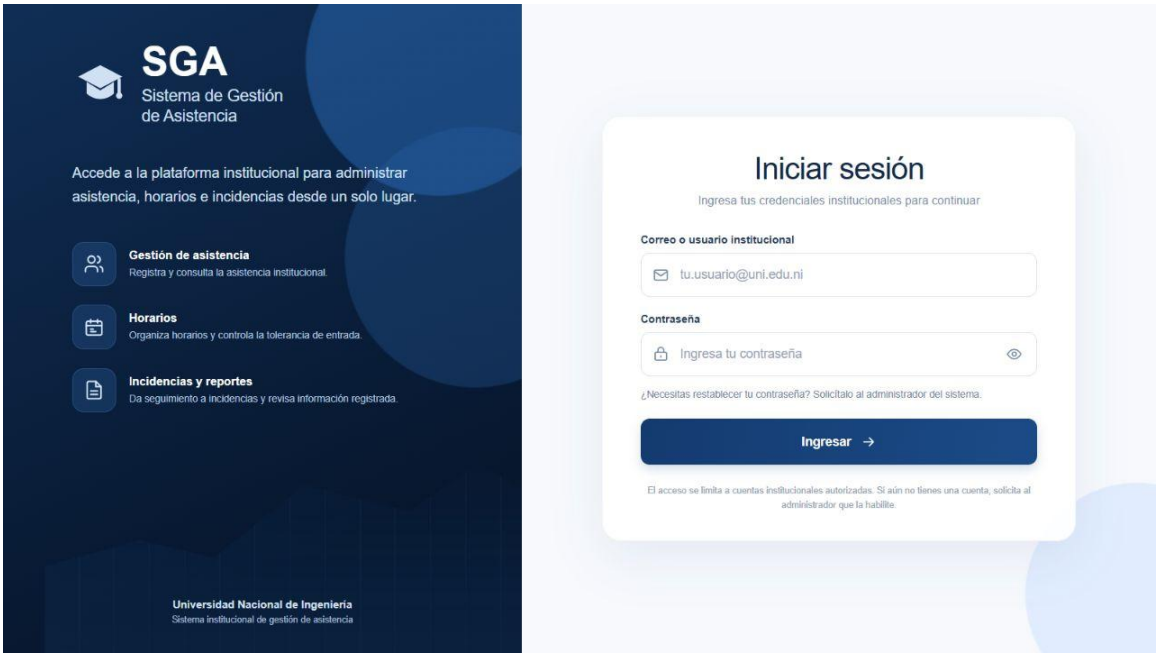


Figura 6.1. Inicio de sesión del SGA publicado en Render el 22 de octubre de 2026.

Paso 7. Abra Personas para registrar código, nombres, apellidos, correo, rol y grupo o área. El código debe ser único. La figura muestra la vista con un filtro sin coincidencias para proteger los datos personales existentes.

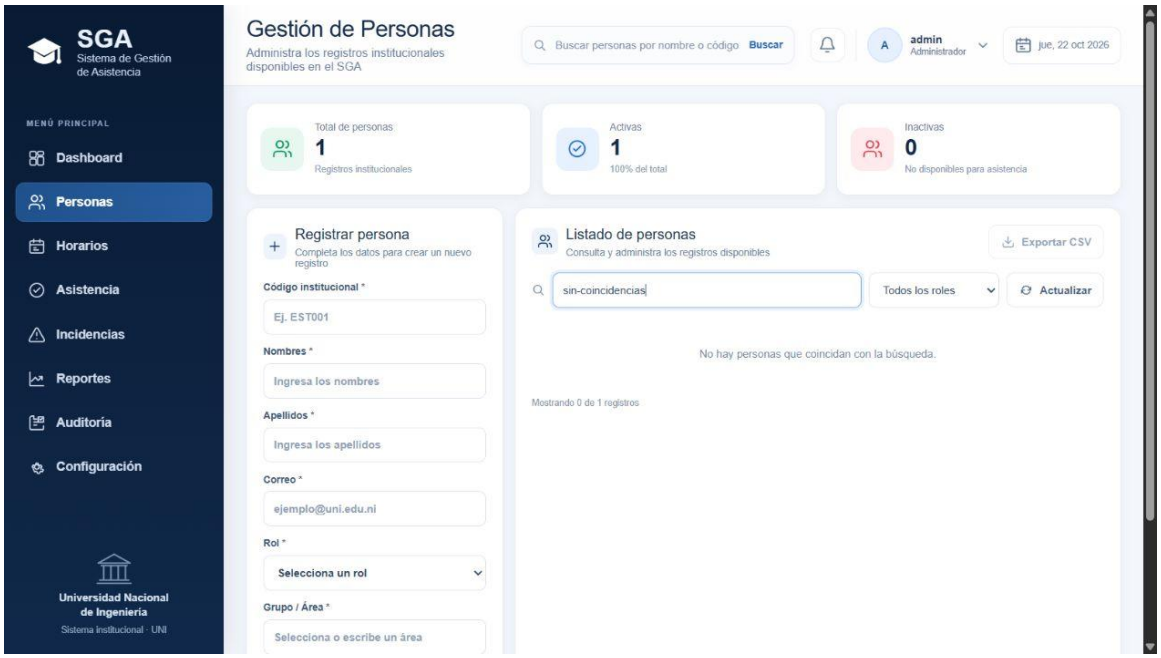


Figura 6.2. Módulo Personas en Render; listado filtrado para ocultar datos personales.

Paso 8. Abra Horarios y registre nombre, tipo, hora de entrada, hora de salida, tolerancia, días aplicables y grupo o área. La hora de salida debe ser posterior a la entrada. La figura muestra el estado inicial, sin horarios guardados.

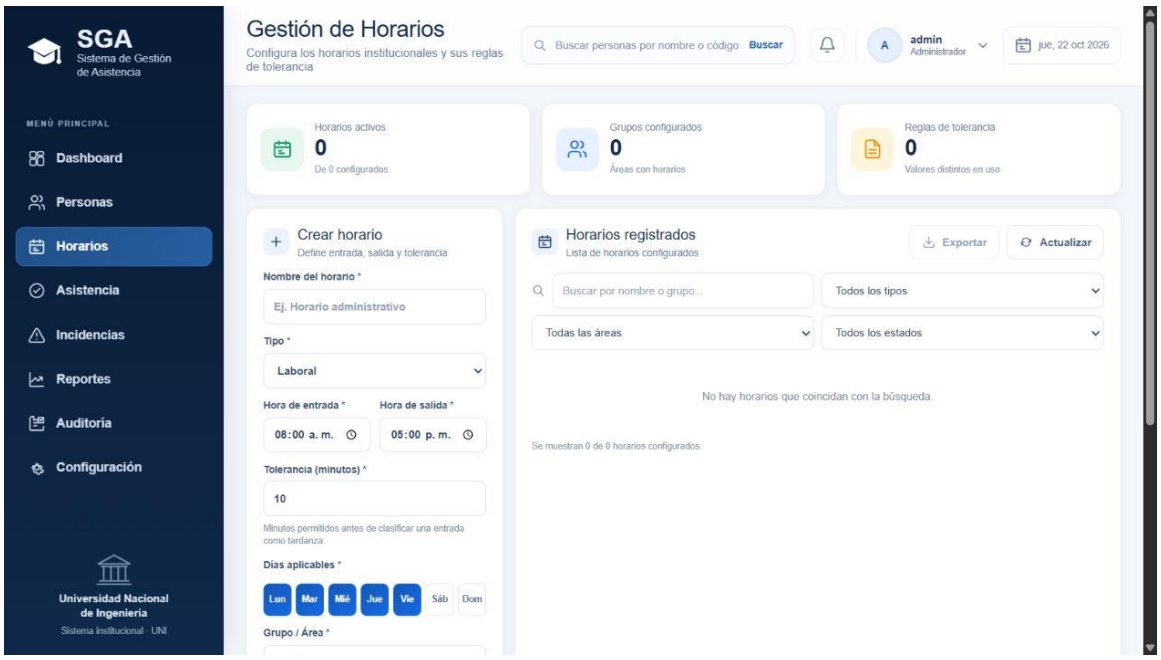


Figura 6.3. Módulo Horarios publicado en Render, antes de crear registros.

Paso 9. Abra Asistencia y seleccione persona, horario, fecha, hora y estado. Primero debe existir al menos un horario y una persona activa; por eso la figura muestra el aviso de configuración inicial. El sistema impide duplicar persona, horario y fecha.

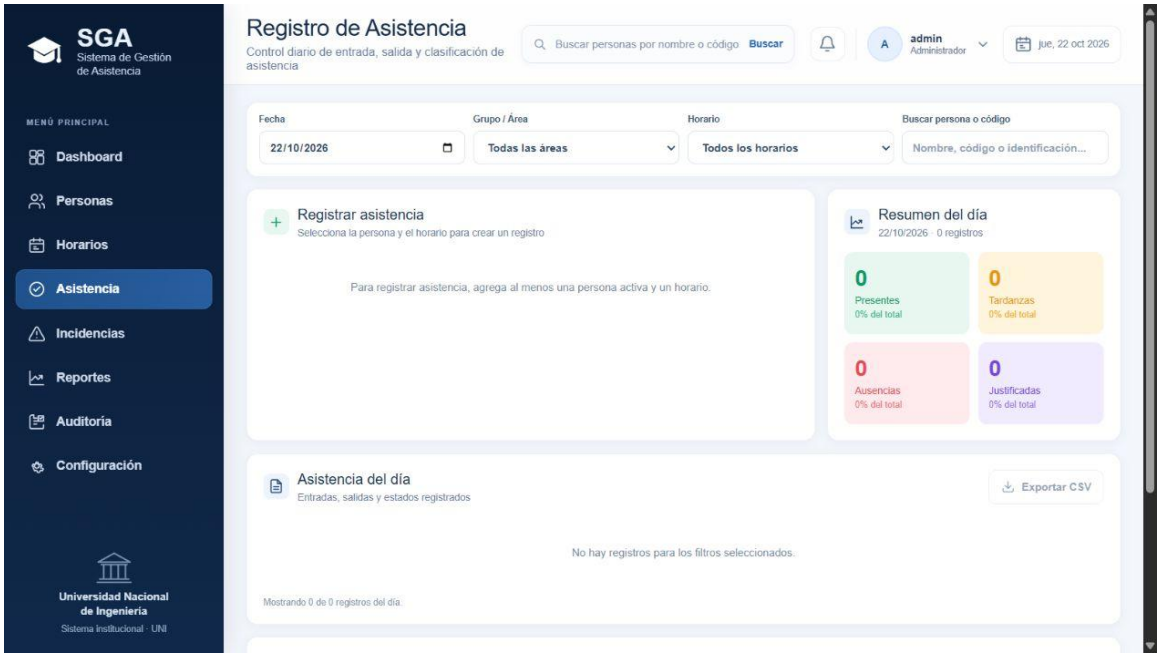


Figura 6.4. Registro de Asistencia en Render, en su estado inicial sin horario.

Paso 10. Abra Incidencias para relacionar una tardanza o ausencia con una asistencia existente. Después podrá registrar y revisar justificaciones según el rol. La figura muestra que aún no hay asistencias ni incidencias en Neon.

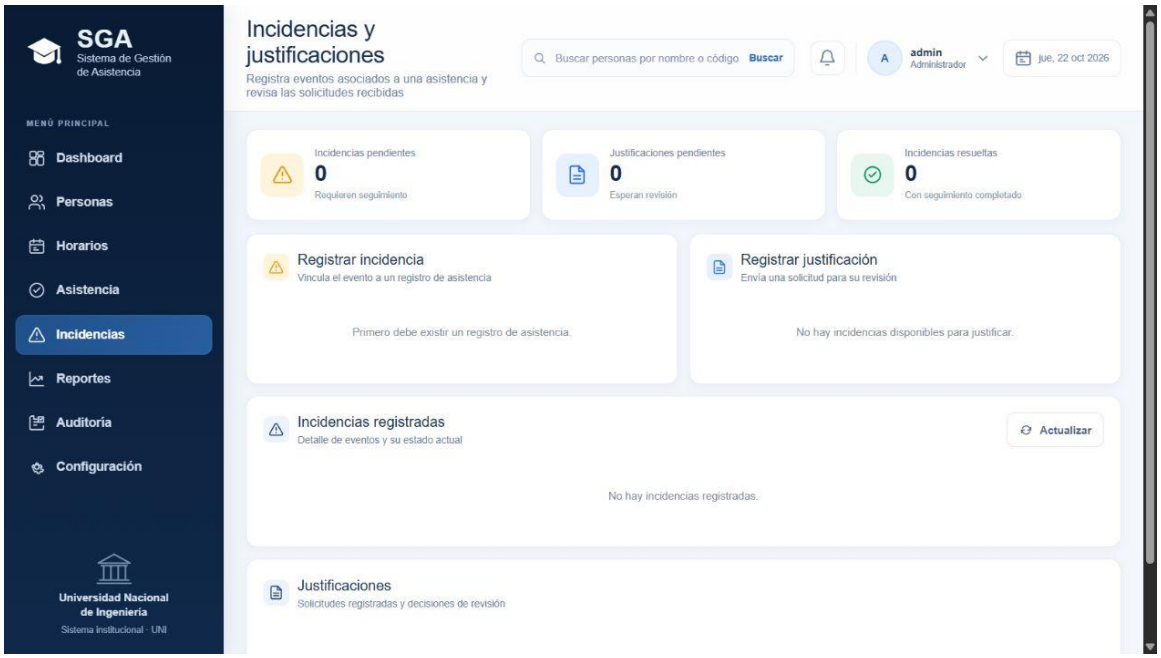


Figura 6.5. Incidencias y justificaciones publicadas en Render, sin registros iniciales.

Paso 11. En Dashboard consulte los conteos y registros recientes. En Reportes seleccione el período, el grupo o área y el tipo de reporte; aplique los filtros y revise los resultados. En estas capturas los indicadores permanecen en cero porque todavía no se cargaron asistencias en Neon.

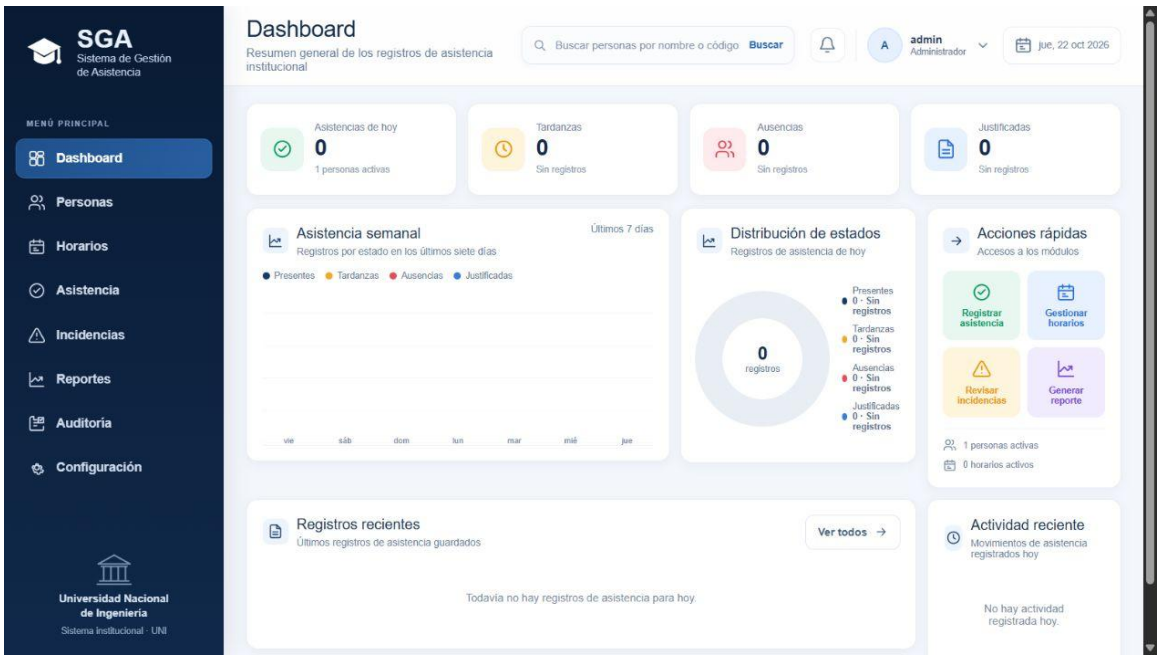


Figura 6.6. Dashboard público del SGA en Render, con la base inicial.

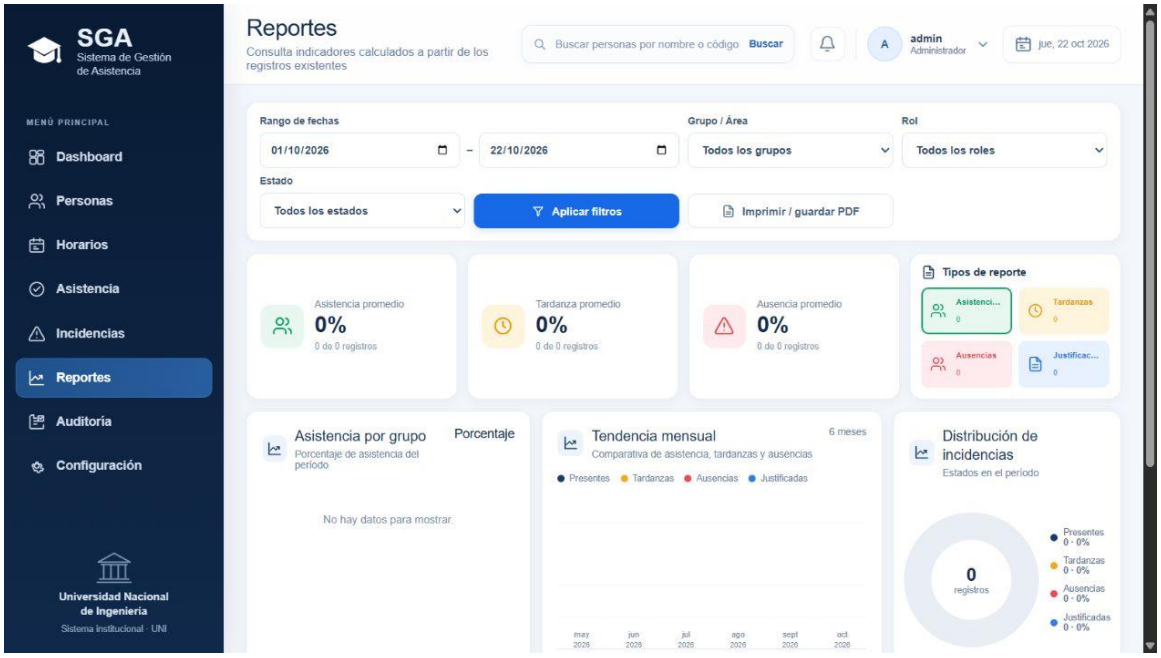


Figura 6.7. Reportes del SGA en Render, antes de registrar asistencias.

Paso 12. Seleccione Cerrar sesión al terminar. La sesión de acceso se elimina del navegador y se revoca el token de renovación.

Supervisor / Docente y Auditor / Dirección pueden consultar los datos autorizados; el servidor rechaza sus operaciones de escritura. Administrador configura cuentas y roles. Administrador y Auditor / Dirección consultan la bitácora; Empleado / Estudiante ve únicamente el historial de su ficha vinculada. Las figuras de este capítulo son capturas de Render; las imágenes del Anexo D son prototipos visuales y no evidencia de ejecución.

CAPÍTULO VII. CONCLUSIONES Y RECOMENDACIONES

El SGA evolucionó de una propuesta de control centralizado a una aplicación web separada en frontend, API REST y persistencia relacional. El código observado contiene módulos de personas, horarios, asistencia, incidencias, justificaciones y reportes, junto con un panel de resumen y documentación OpenAPI/Swagger.

En la Fase 3 se implementaron autenticación con JWT en cookies HttpOnly, protección CSRF, renovación y revocación de sesión, limitación de intentos, encabezados de seguridad y permisos por rol. También se incorporaron auditoría, configuración de cuentas, vínculo entre cuenta y Persona e historial personal de asistencia. Docker Compose se verificó localmente; la aplicación está publicada en Render y utiliza PostgreSQL de Neon.

El proyecto ya dispone de URL pública, API accesible, capturas de producción en el manual y cambios sincronizados con GitHub. Persisten tareas antes de una adopción institucional: pruebas funcionales, de carga y de seguridad documentadas, piloto con una unidad de la UNI y respaldos con restauración comprobada. La auditoría, la asociación de cuentas con personas y el historial personal están implementados, pero requieren validación con usuarios y reglas organizativas.

Recomendaciones prioritarias

R1. Confirmar con la UNI una unidad piloto, persona responsable, proceso actual, finalidad de los datos y reglas de acceso.

R2. Revisar el código de seguridad en equipo y ejecutar la matriz de pruebas funcionales, permisos, CSRF, HTTPS, CORS, carga y restauración; conservar evidencias fechadas.

R3. Vigilar los límites gratuitos de Render y Neon, documentar el procedimiento de respaldo y restauración, y prever alojamiento de mayor disponibilidad si la UNI decide usar el sistema de forma continua.

R4. Vincular cuentas con personas y definir el alcance por área antes de habilitar autoservicio o ampliar acceso a datos institucionales.

R5. Validar con la unidad piloto la configuración de roles, la bitácora de auditoría y el historial personal; definir retención y alcance organizativo antes de habilitar la adopción institucional.

R6. Sincronizar los cambios con el repositorio, registrar contribuciones verificables de los tres integrantes y preparar una defensa técnica de 15 minutos, conforme a la guía (Universidad Nacional de Ingeniería, 2026a).

REFERENCIAS

Django REST Framework. (s. f.-a). *Permissions*. Recuperado el 21 de octubre de 2026, de https://www.django-rest-framework.org/api-guide/permissions/

Django REST Framework. (s. f.-b). *Throttling*. Recuperado el 21 de octubre de 2026, de https://www.django-rest-framework.org/api-guide/throttling/

Django REST Framework Simple JWT. (s. f.-a). *Blacklist app*. Recuperado el 21 de octubre de 2026, de https://django-rest-framework-simplejwt.readthedocs.io/en/latest/blacklist_app.html

Django REST Framework Simple JWT. (s. f.-b). *Settings*. Recuperado el 21 de octubre de 2026, de https://django-rest-framework-simplejwt.readthedocs.io/en/latest/settings.html

Django Software Foundation. (s. f.-a). *Deployment checklist*. Recuperado el 21 de octubre de 2026, de https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

Django Software Foundation. (s. f.-b). *Security in Django 5.2*. Recuperado el 21 de octubre de 2026, de https://docs.djangoproject.com/en/5.2/topics/security/

Equipo SGA. (2026a). *Fase 1 y 2 SGA (V2): diagnóstico, formulación y avances del proyecto* [Documento de trabajo facilitado por el equipo docente].

Equipo SGA. (2026b). Repositorio del proyecto Sistema de Gestión de Alerta [Repositorio de código]. GitHub. Recuperado el 22 de octubre de 2026, de https://github.com/CalMiranda/SGA

Equipo SGA. (2026c). *Retroalimentación docente* [Documento de trabajo facilitado por el equipo].

Universidad Nacional de Ingeniería. (s. f.). *Logotipo institucional* [Archivo de imagen]. Recuperado el 21 de octubre de 2026, de https://www.uni.edu.ni/assets/images/logouni12.png

Universidad Nacional de Ingeniería. (2026a). *Guía metodológica de evaluación del proyecto de Diseño de Sistemas en Internet* [Guía de curso facilitada por el equipo docente].

ANEXOS

ANEXO A. Roles y matriz de permisos

Rol	Lectura API	Escritura API	Condición / límite
Administrador	Sí	Sí	Grupo de Django; el superusuario tiene acceso total.
Gestor de asistencia	Sí	Sí	Opera los recursos de negocio.
Supervisor / Docente	Sí	No	Solo lectura; no se filtra por grupo o área.
Auditor / Dirección	Sí	No	Solo lectura; consulta de bitácora autorizada.
Empleado / Estudiante	Sí, solo personal	No	Consulta únicamente /api/mi-historial/ para la ficha vinculada; no accede a otros registros.

La autorización de los recursos de negocio se aplica en la API. Ocultar o mostrar controles en una interfaz no sustituye la validación del servidor. Los permisos y el alcance deben revisarse con la unidad piloto antes de dar acceso a datos reales.

ANEXO B. Matriz propuesta de pruebas

Caso	Precondición	Resultado esperado	Estado
T-01 Login correcto	Usuario activo y grupo habilitado.	200, cookies HttpOnly y acceso al Dashboard.	Pendiente.
T-02 Clave incorrecta	Usuario habilitado.	401 genérico; no se emite JWT.	Pendiente.
T-03 Usuario sin rol	Credenciales válidas, sin grupo permitido.	403; sin cookies de sesión.	Pendiente.
T-04 Acceso anónimo	Sin cookies.	API de negocio rechaza la solicitud.	Pendiente.
T-05 Solo lectura	Supervisor autenticado.	GET permitido; POST/PATCH/DELETE rechazado.	Pendiente.
T-06 Duplicidad	Persona y asistencia ya registrados.	Validación impide código o jornada duplicados.	Pendiente.
T-07 CSRF	Cookie de sesión sin cabecera CSRF válida.	Método inseguro rechazado.	Pendiente.
T-08 HTTPS	Aplicación publicada bajo dominio TLS.	Cookies Secure y redirección a HTTPS.	Pendiente.
T-09 Restauración	Respaldo PostgreSQL disponible.	Base restaurada y registros comprobados.	Pendiente.

Caso	Precondición	Resultado esperado	Estado
T-10 Configuración	Cuenta Administrador y cuenta de otro rol.	Solo Administrador gestiona cuentas; se registra el cambio y se revocan sesiones cuando corresponde.	Pendiente.
T-11 Auditoría	Cuentas Administrador, Auditor y Gestor.	Administrador y Auditor consultan; otros roles no acceden; ningún rol modifica o elimina eventos.	Pendiente.
T-12 Historial propio	Dos cuentas Empleado vinculadas a fichas diferentes.	Cada cuenta recibe solo la asistencia e incidencias de su ficha en /api/mi-historial/.	Pendiente.
T-13 Vínculo activo	Cuenta Empleado sin ficha o vinculada a una ficha inactiva.	El inicio de sesión o el acceso al historial se rechaza según el estado de la ficha.	Pendiente.
T-14 Revocación	Cuenta con sesión previa y token de renovación vigente.	Cambiar contraseña, rol o estado revoca la renovación de sesión anterior.	Pendiente.

Registrar por caso: versión desplegada, fecha, responsable, datos de prueba, pasos, salida observada, evidencia y corrección asociada. No adjuntar datos personales reales en capturas de defensa.

ANEXO C. Diagramas UWE — C.1. Casos de uso de procesos

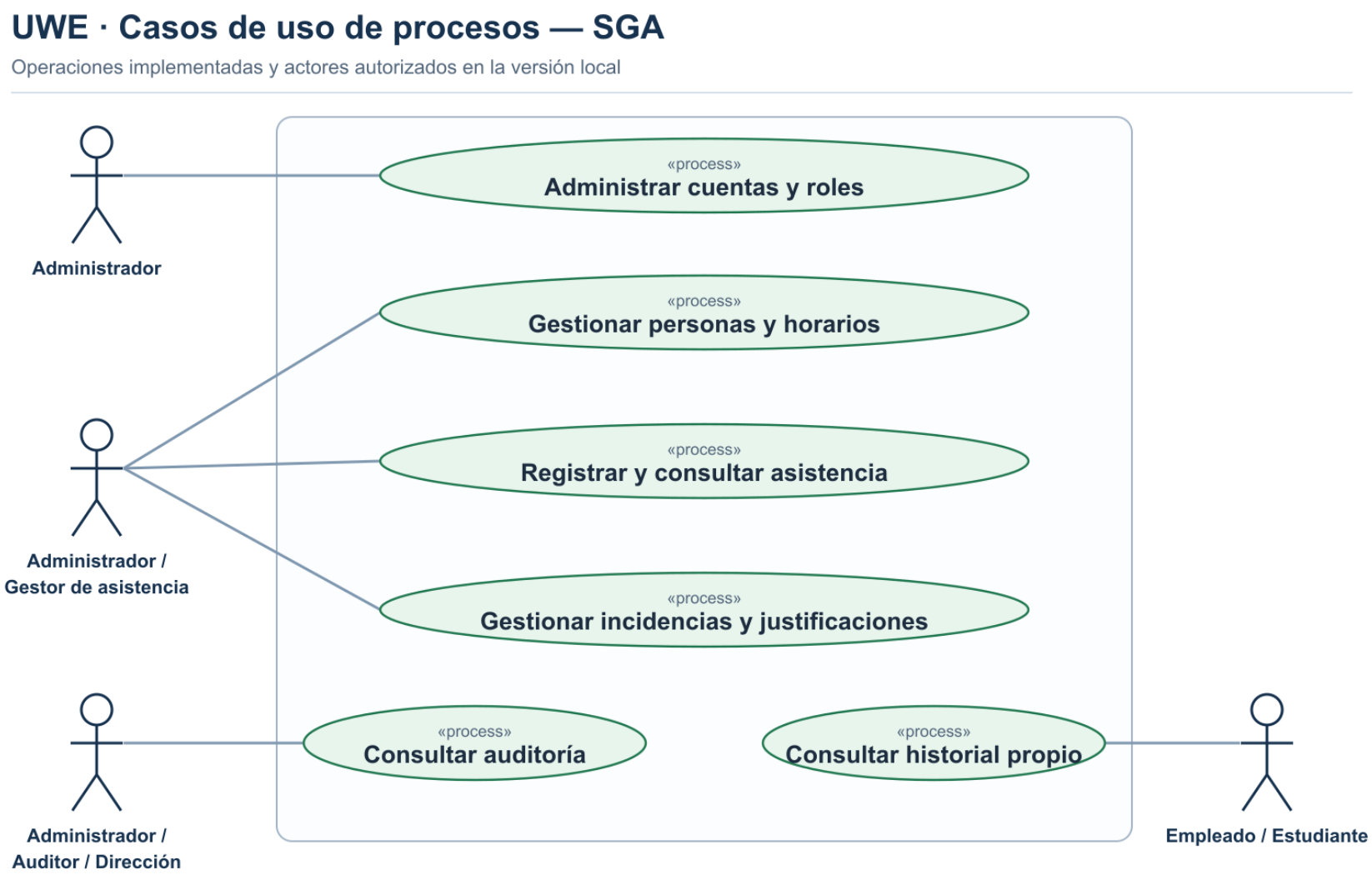


Figura C.1. Casos de uso de procesos implementados y permisos de sus actores en el SGA.

C.2. Casos de uso navegacionales

UWE · Casos de uso navegacionales — SGA

Módulos y permisos de consulta o gestión por rol

Rol de acceso	Dashboard	Personas	Horarios	Asistencia	Incidencias	Reportes	Auditoría	Configuración	Mi historial
Administrador	C	G	G	G	G	C	C	G	
Gestor de asistencia	C	G	G	G	G	C			
Supervisor / Docente	C	C	C	C	C	C			
Auditor / Dirección	C	C	C	C	C	C	C		
Empleado / Estudiante									C

G Gestiona y modifica los datos permitidos

C Consulta sin modificar

Las rutas requieren autenticación. El servidor aplica estos permisos en cada solicitud.

Figura C.2. Casos navegacionales por rol. G = gestiona; C = consulta.

C.3. Modelo de contenido

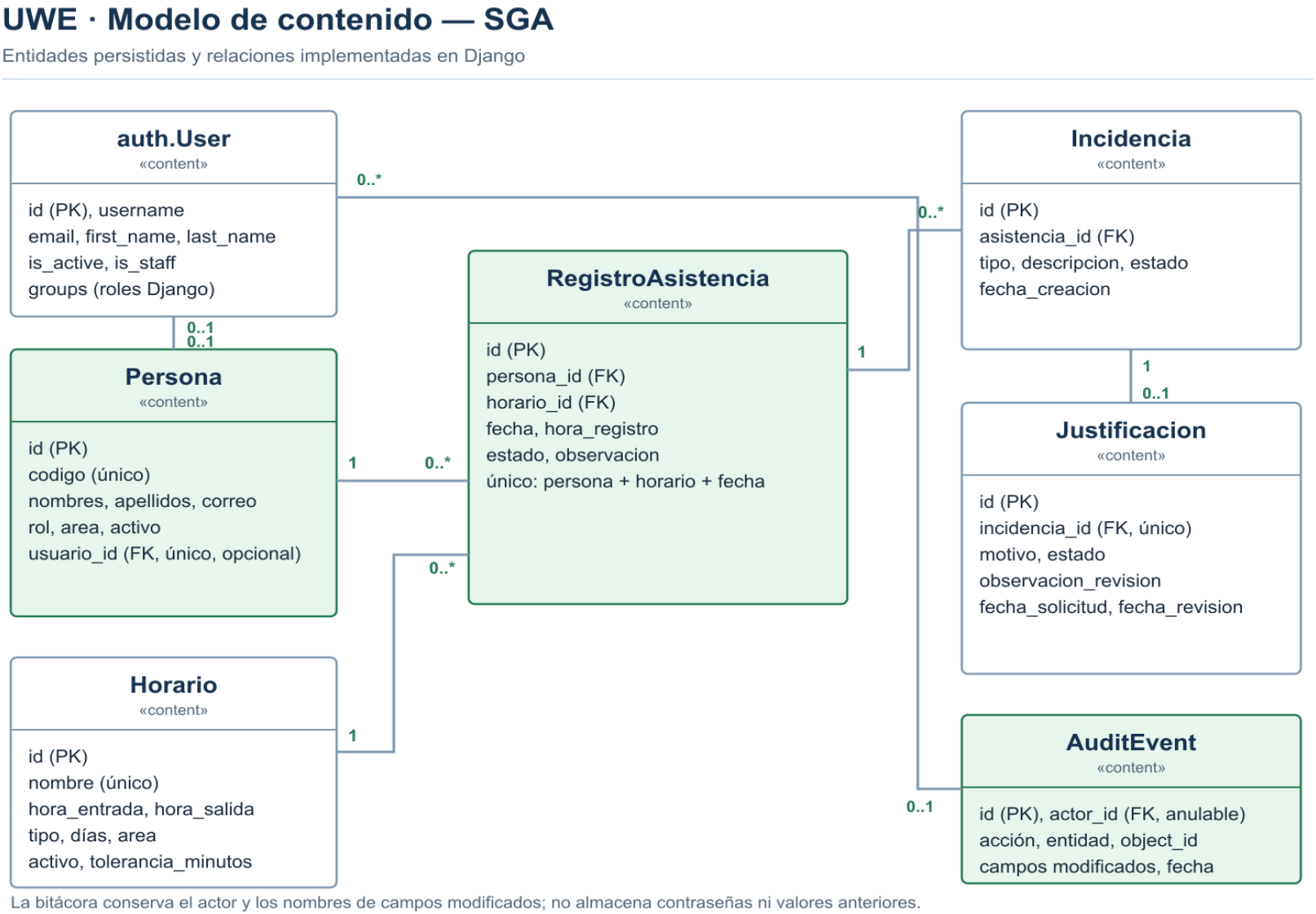


Figura C.3. Modelo de contenido con la relación opcional entre la cuenta y Persona y la bitácora de auditoría.

C.4. Modelo de navegación

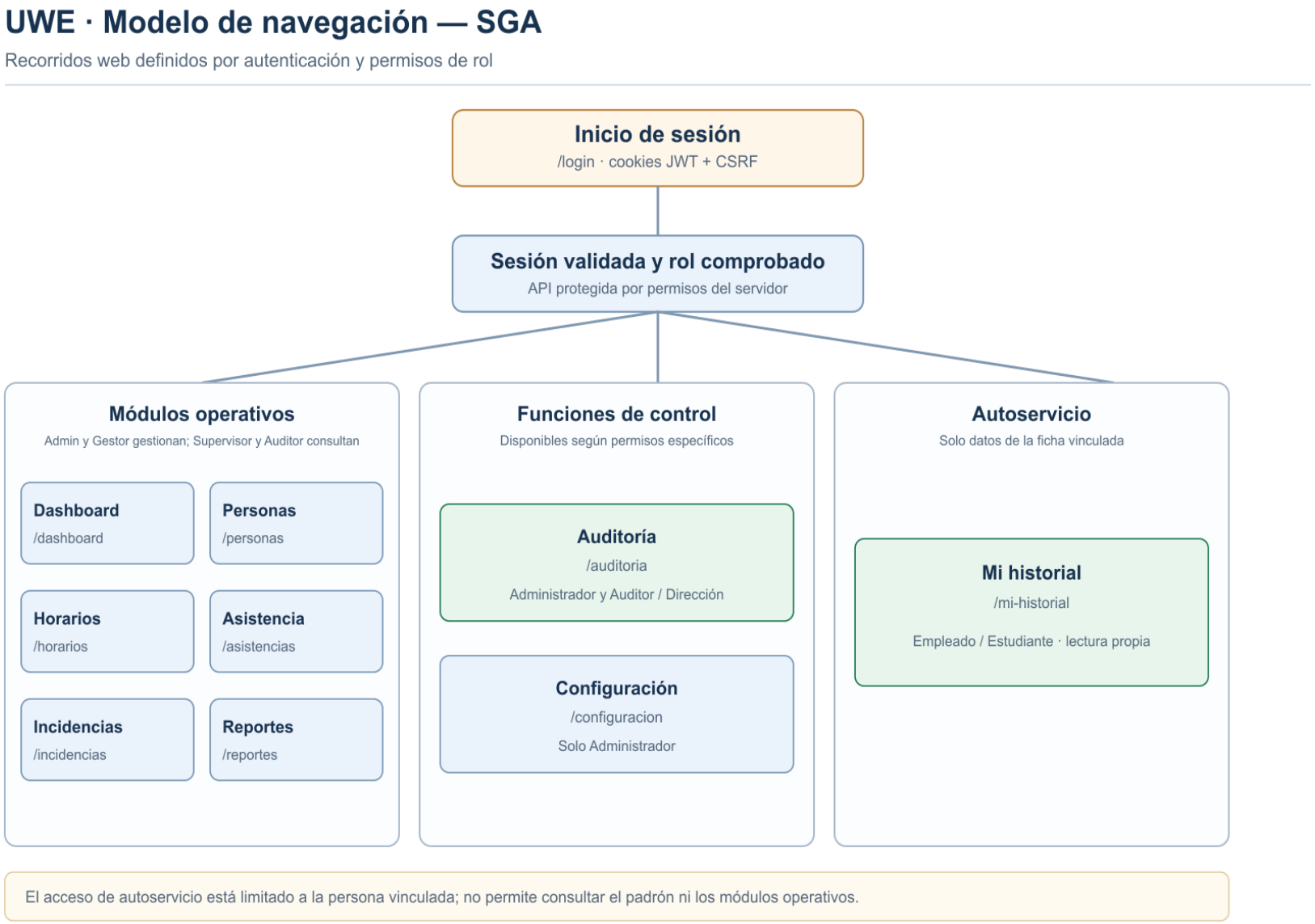
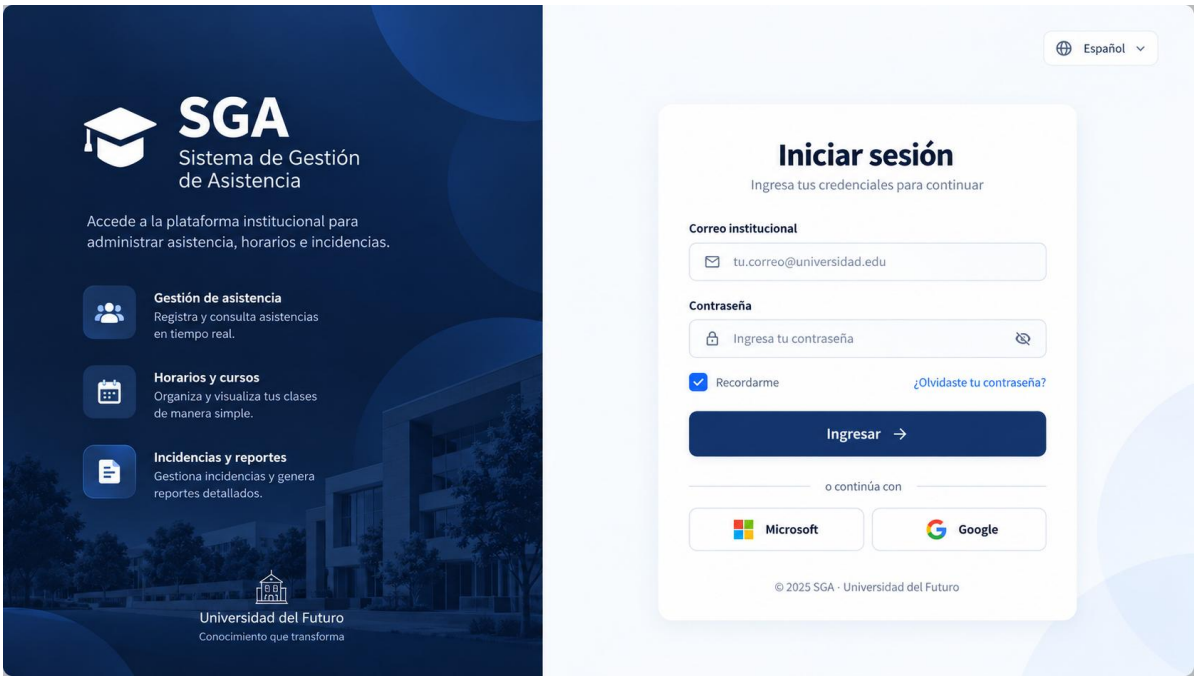


Figura C.4. Mapa de navegación implementado para módulos operativos, administración, auditoría y autoservicio.

ANEXO D. Prototipos visuales recibidos

Estas imágenes se presentan como material de diseño. No son capturas reales de la versión desplegada ni evidencia de las pruebas requeridas por la guía. Los textos, métricas, marcas y controles pueden ser ilustrativos o diferir del código actual.

D.1. Inicio de sesión



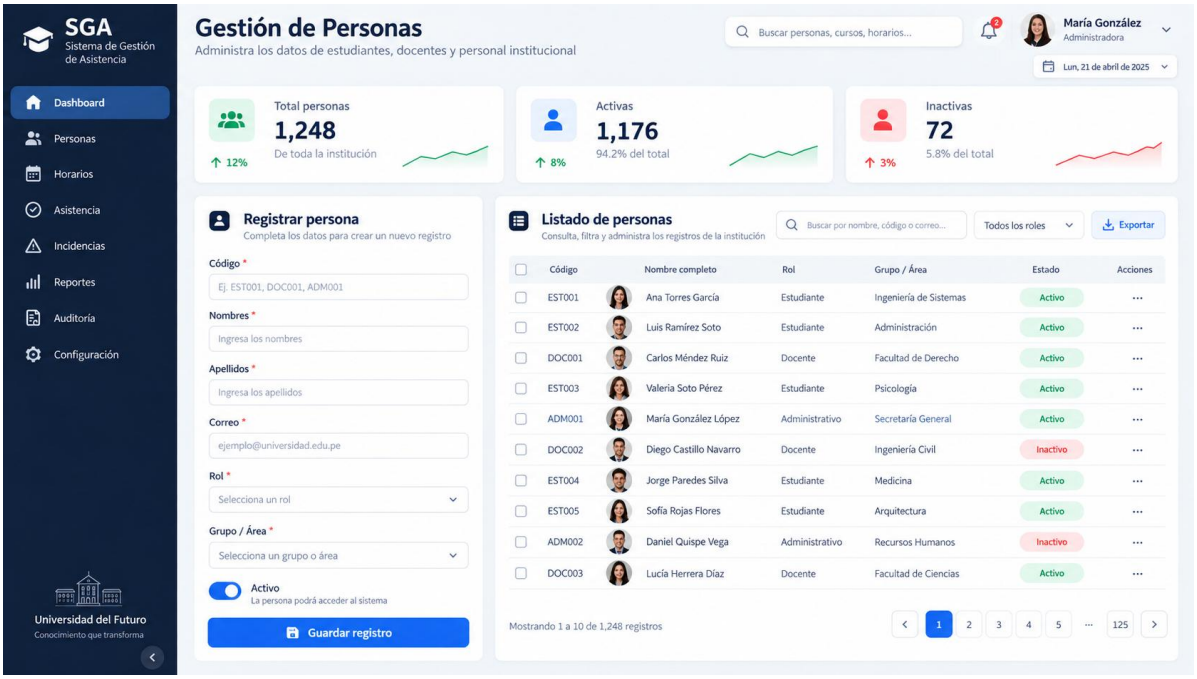
Prototipo visual del avance del proyecto; datos ilustrativos; no corresponde a evidencia de producción.

D.2. Dashboard



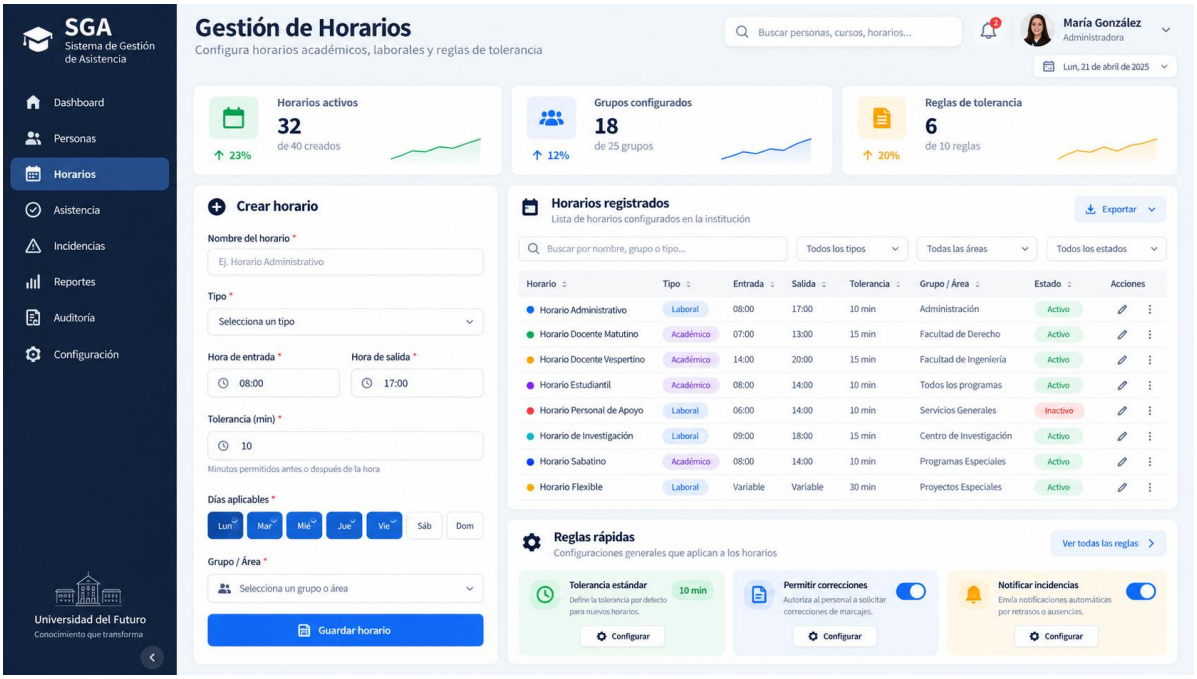
Prototipo visual del avance del proyecto; datos ilustrativos; no corresponde a evidencia de producción.

D.3. Gestión de personas



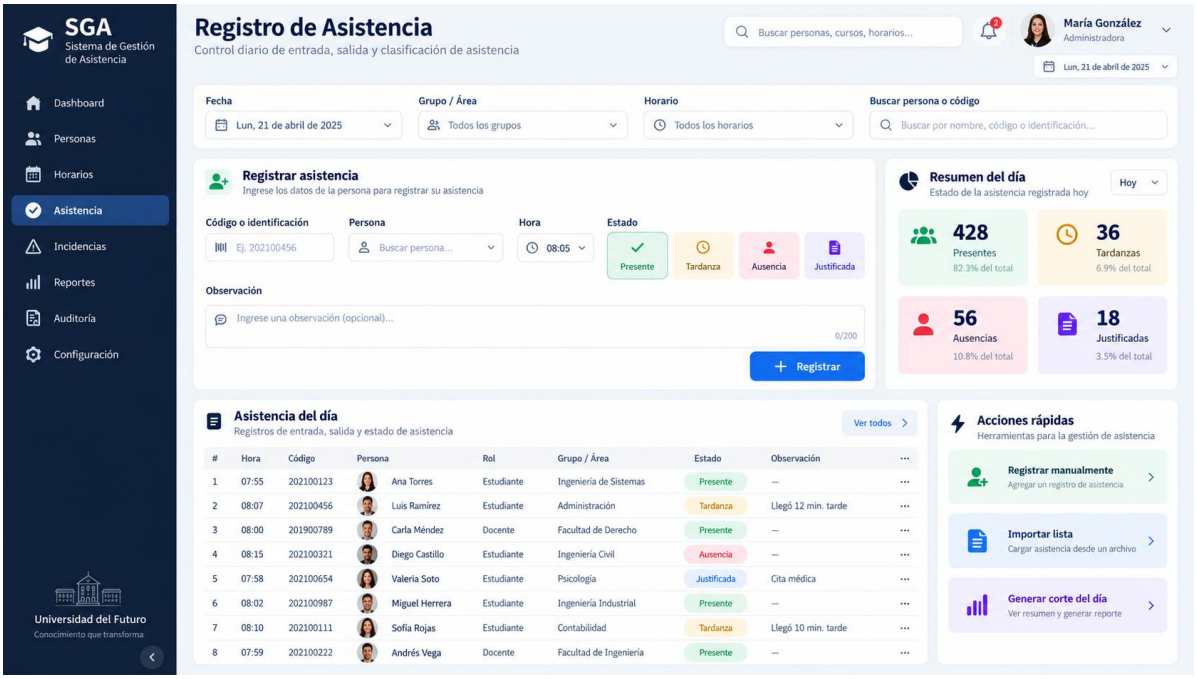
Prototipo visual del avance del proyecto; datos ilustrativos; no corresponde a evidencia de producción.

D.4. Horarios



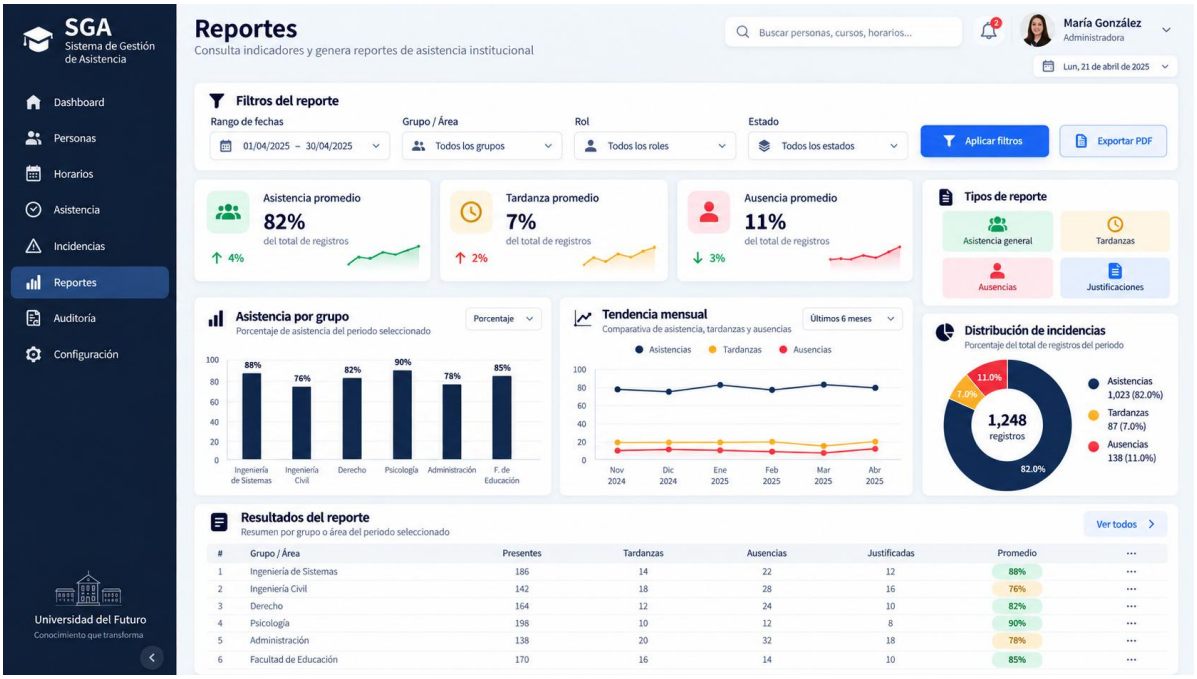
Prototipo visual del avance del proyecto; datos ilustrativos; no corresponde a evidencia de producción.

D.5. Registro



Prototipo visual del avance del proyecto; datos ilustrativos; no corresponde a evidencia de producción.

D.6. Reportes



Prototipo visual del avance del proyecto; datos ilustrativos; no corresponde a evidencia de producción.

ANEXO E. Enlaces, API y despliegue Docker

Documentación pública de API: https://sga-cal-miranda-web.onrender.com/api/docs/ (Swagger UI) y https://sga-cal-miranda-web.onrender.com/api/schema/ (OpenAPI). Las direcciones locales http://localhost:8000/api/docs/ y http://localhost:8000/api/schema/ permanecen disponibles cuando el backend Docker está activo.

Acción de preparación	Descripción
Configurar variables	Copiar .env.example como .env en la raíz; reemplazar dominio, origen público, SECRET_KEY y contraseñas por valores aleatorios únicos.
Preparar DNS	Apuntar el dominio elegido al servidor y permitir acceso entrante a 80/443 para que Caddy gestione HTTPS.
Levantar servicios	Desde SGA-main ejecutar docker compose up --build -d; el backend aplica migraciones al iniciar.
Crear administrador	Ejecutar docker compose exec backend python manage.py createsuperuser; luego asignar el grupo adecuado en /admin/.
Operar y mantener	Revisar logs, respaldos y actualización de imágenes; ensayar restauración antes de usar datos institucionales.

La publicación actual utiliza Render y Neon con las variables privadas DATABASE_URL, DJANGO_SECRET_KEY, DJANGO_ALLOWED_HOSTS, CORS_ALLOWED_ORIGINS y CSRF_TRUSTED_ORIGINS. El archivo .env.example conserva un dominio ilustrativo para despliegues propios; no representa la URL actual. No se deben publicar secretos ni credenciales en el repositorio (Django Software Foundation, s. f.-a).

E.1. Enlaces del proyecto y documentación

- Repositorio de código y cambios de Fase 3: https://github.com/CalMiranda/SGA.
- Aplicación pública: https://sga-cal-miranda-web.onrender.com/login. Swagger UI: https://sga-cal-miranda-web.onrender.com/api/docs/.
- Esquema OpenAPI: https://sga-cal-miranda-web.onrender.com/api/schema/. No se encontró una colección Postman en el proyecto.

E.2. Endpoints principales de autenticación

Verbo HTTP	Ruta	Propósito

Verbo HTTP	Ruta	Propósito
GET	/api/auth/csrf/	Obtiene el token CSRF para el cliente.
POST	/api/auth/login/	Valida las credenciales e inicia sesión.
POST	/api/auth/refresh/	Renueva la sesión mediante cookie segura.
POST	/api/auth/logout/	Cierra sesión y revoca el token de renovación.
GET	/api/auth/me/	Devuelve los datos del usuario autenticado y sus permisos.
GET	/api/auditoria/	Consulta filtrada de eventos; solo Administrador y Auditor / Dirección.
GET, POST, PATCH	/api/configuracion/usuarios/ y /api/configuracion/usuarios/{id}/	Lista, crea y modifica cuentas; solo Administrador.
GET	/api/configuracion/roles/ y /api/configuracion/personas/	Consulta roles disponibles y fichas activas para vincular; solo Administrador.
GET	/api/mi-historial/	Consulta asistencia e incidencias propias de la ficha vinculada.

E.3. Archivos fuente relevantes

- backend/config/urls.py — rutas de autenticación, API, esquema OpenAPI y Swagger.
- backend/config/auth_views.py, authentication.py, permissions.py y backend/gestion/ — sesión JWT en cookies, CSRF, permisos por rol, auditoría y endpoints de configuración e historial.
- backend/config/settings.py — configuración de seguridad y conectividad por variables de entorno; no publicar valores secretos.
- backend/personas/, backend/horarios/, backend/asistencias/ y backend/incidencias/ — modelos, vistas, serializadores y rutas de negocio.
- frontend/src/app/auditoria, /configuracion, /mi-historial, frontend/src/app/login/page.tsx y frontend/src/lib/api.ts — módulos de auditoría, cuentas, autoservicio, inicio de sesión y cliente de API.
- render.yaml y docs/despliegue-render-neon.md — infraestructura pública en Render y Neon; docker-compose.local.yml, docker-compose.yml y Caddyfile — ejecución local o alternativa con dominio propio.